DOMAIN DRIVEN DESIGN - JAVA - GLOBAL SOLUTION 2026



NexRain

Documentação do Projeto

Equipe NexRain

Mateus Saavedra De Mendonça Cruz - RM563266

Davi Melo - RM561818

Danilo Fernandes - RM561657

FIAP - Engenharia de Software - Global Solution 2026 - CitizenTech

Sumário

1. Descritivo da Solução 3

2. Justificativa 3

3. Objetivos 4

4. Principais Funcionalidades 4

5. Organização do Projeto 5

6. Tabela de Endpoints 6

    6.1 Exemplos de requisição 8

7. Protótipos de Tela 9

8. Modelo do Banco de Dados (DER) 13

9. Diagrama de Classes 14

10. Como Executar 18

1. Descritivo da Solução

O NexRain é uma API RESTful em Java com Spring Boot que monitora alagamentos urbanos em São Paulo. A API analisa imagens de câmeras da cidade, registra as ocorrências de alagamento no banco Oracle da FIAP, calcula o risco de alagamento de cada região com base na previsão de chuva e sugere rotas de carro que desviam das ruas alagadas.

A solução segue as quatro etapas definidas pelo grupo:

- **Monitoramento por IA:** a API recebe a imagem de uma câmera (ou a foto enviada por um cidadão) e envia para a API da OpenAI (GPT-4o-mini), que informa se há alagamento, a altura estimada da água e se a via é transitável. Sem a chave da OpenAI, a análise é feita por um motor próprio que avalia a cor e a textura da imagem.
- **Análise e registro:** quando há alagamento, a API grava uma ocorrência com local, horário, altura da água, nível de risco e confiança. A ocorrência começa como pendente e pode ser confirmada, descartada ou resolvida por um operador.
- **Alertas:** ao confirmar uma ocorrência de risco alto ou crítico, a API cria um alerta para a região. Também é possível gerar alertas a partir da previsão de chuva.
- **Mobilidade:** a API consulta o OSRM para obter as rotas entre dois pontos, verifica quais passam perto de um alagamento ativo e escolhe a mais segura, criando pontos de desvio quando necessário.

Os dados de previsão vêm da API Open-Meteo, com a API do MET Norway como reserva. As câmeras ao vivo são as câmeras públicas da CET-SP e, para demonstrar a detecção, também foram cadastradas imagens de enchentes reais do Wikimedia Commons. A API possui ainda um modo de chuva simulada para testar o cálculo de risco em dias sem chuva.

2. Justificativa

Os alagamentos acontecem com frequência em São Paulo durante as chuvas de verão e afetam vias importantes como a Av. 23 de Maio, a região da Praça da Bandeira e a Av. Pompeia. Além do trânsito, há risco de perda de veículos e de acidentes com pessoas que tentam atravessar ruas alagadas.

Hoje as informações estão espalhadas: a previsão do tempo é dada para a cidade inteira, os alertas não indicam quais ruas evitar e os aplicativos de trânsito só percebem o problema depois que os carros já pararam. A cidade já possui câmeras instaladas que podem ser usadas para identificar os pontos de alagamento.

Por isso o projeto junta a análise das câmeras, a previsão de chuva e o cálculo de rotas em uma única API, que pode ser consumida por sites, aplicativos ou pelos sistemas da Defesa Civil.

3. Objetivos

Objetivo geral

Desenvolver uma API RESTful em Java com Spring Boot e banco Oracle que represente o domínio de monitoramento de alagamentos urbanos, permitindo identificar, registrar, validar e comunicar os riscos e sugerir rotas seguras.

Objetivos específicos

- Analisar imagens de câmeras urbanas e estimar a altura da água e o nível de risco.
- Registrar as ocorrências no Oracle e controlar a validação (pendente, confirmada, descartada e resolvida).
- Calcular um índice de risco de 0 a 100 por região usando a previsão de chuva, o limite de chuva do bairro, o histórico de alagamentos e as ocorrências ativas.
- Gerar alertas automáticos sem repetir alertas para a mesma região.
- Calcular rotas que evitam os pontos alagados e comparar com a rota mais rápida.
- Disponibilizar o CRUD completo (GET, POST, PUT e DELETE) de regiões, câmeras, ocorrências, alertas e rotas, com validações e tratamento de erros.
- Publicar a API no Render.

4. Principais Funcionalidades

- **Análise de imagem:** câmera cadastrada ou foto em base64; usa a OpenAI e, se ela não responder, o motor local. Uma câmera com ocorrência ativa tem a ocorrência atualizada em vez de criar outra.
- **Índice de risco por região:** regra da classe `MonitorRisco`, explicada na tabela abaixo.
- **Validação das ocorrências:** as regras de mudança de status ficam na entidade `Ocorrencia`. Por exemplo, não é possível confirmar uma ocorrência que já foi resolvida.
- **Rota segura:** classe `PlanejadorRota`. O custo de cada caminho é o tempo de viagem mais uma penalidade para cada alagamento no trajeto. Quando a melhor rota ainda passa por um alagamento, são criados pontos de desvio dos dois lados da rua e o OSRM é consultado de novo.
- **Alertas:** criados ao confirmar uma ocorrência grave ou a partir da previsão de chuva.
- **CRUD completo** das cinco tabelas, com validações nos services e respostas de erro padronizadas.

Cálculo do índice de risco (MonitorRisco)

Fator	Pontos (máx.)
Maior chuva prevista nas próximas 6h comparada com o limite de chuva do bairro	50
Chuva acumulada nas próximas 3h	15
Probabilidade de chuva	10
Suscetibilidade do bairro (ALTA 20, MEDIA 10, BAIXA 0; metade se não houver chuva)	20
Ocorrências ativas na região (crítica 30, alta 20, média 10, baixa 4)	35

Índice até 24 = BAIXO, 25 a 49 = MÉDIO, 50 a 74 = ALTO e a partir de 75 = CRÍTICO. Para a altura da água: abaixo de 10 cm BAIXO, de 10 a 29 cm MÉDIO, de 30 a 49 cm ALTO e a partir de 50 cm CRÍTICO.

5. Organização do Projeto

O projeto segue a mesma organização dos projetos anteriores da disciplina: controller, service e DAO, com as regras de negócio nas classes do pacote `entities`.

Pacote	Conteúdo
`entities`	`Regiao`, `Camera`, `Ocorrencia`, `Alerta`, `Rota`, `Coordenada` (distâncias entre pontos) e as regras `MonitorRisco` e `PlanejadorRota`.
`services`	Validações e regras de cada recurso, monitoramento, risco, rotas e as chamadas às APIs externas (Open-Meteo, MET Norway, OSRM, Nominatim e OpenAI).
`dao`	Acesso ao Oracle com JDBC (`PreparedStatement`), um DAO por tabela.
`controller`	Endpoints REST, configuração de CORS e o `TratadorExcecoes`.
`api` / `dto`	Dados recebidos das APIs externas e objetos de entrada e saída da API.
`conexoes`	`ConexaoFactory`, `Configuracao` (lê as variáveis de ambiente ou o `local.properties`) e `ClienteHttp`.
`excecoes`	Exceções de regra de negócio, recurso não encontrado e falha em serviço externo.
`main`	Testes pelo console: `TesteConexao`, `TesteSelecionarRegiao` e `TesteMonitorRisco`.

Integrações

Serviço	Uso na API	Chave
Oracle FIAP	Banco de dados (5 tabelas)	usuário e senha
Open-Meteo	Previsão de chuva por hora	não
MET Norway	Previsão reserva, usada se a Open-Meteo falhar	não
OSRM	Rotas de carro	não
Nominatim (OpenStreetMap)	Busca de endereços	não
OpenAI (GPT-4o-mini)	Análise das imagens	sim (opcional)
CET-SP	Imagens das câmeras de trânsito ao vivo	não

6. Tabela de Endpoints

Base local: `http://localhost:8080`. Base publicada: `https://nexrain-springboot.onrender.com`. Requisições e respostas em JSON.

Verbo	URI	Descrição	Status
Monitoramento, previsão e mapas			
POST	`/api/monitoramento/analises`	Analisa a imagem de uma câmera ou foto (visão computacional) e registra a ocorrência	200, 201, 400, 404, 502, 500
GET	`/api/riscos?simulacao=false`	Risco de alagamento de cada região (previsão real ou temporal simulado)	200, 502, 500
GET	`/api/clima?latitude=&longitude=`	Clima atual e chuva prevista hora a hora de um ponto	200, 400, 502
GET	`/api/geocodificacao?endereco=`	Busca coordenadas de um endereço (OpenStreetMap)	200, 400, 502
GET	`/api/painel`	Números do painel (regiões, câmeras, ocorrências, alertas, rotas)	200, 500
GET	`/api/health`	Verifica se a API está no ar e qual motor de IA está ativo	200
Regiões			
GET	`/api/regioes`	Lista as regiões monitoradas	200, 500
GET	`/api/regioes/{id}`	Busca uma região pelo id	200, 400, 404, 500
GET	`/api/regioes/{id}/cameras`	Câmeras da região	200, 400, 404, 500
GET	`/api/regioes/{id}/ocorrencias`	Histórico de ocorrências da região	200, 400, 404, 500
POST	`/api/regioes`	Cadastra uma região	201, 400, 500
PUT	`/api/regioes/{id}`	Atualiza uma região	200, 400, 404, 500
DELETE	`/api/regioes/{id}`	Exclui uma região (e suas câmeras, ocorrências e alertas)	204, 400, 404, 500
Câmeras			
GET	`/api/cameras`	Lista as câmeras	200, 500
GET	`/api/cameras/{id}`	Busca uma câmera pelo id	200, 400, 404, 500
POST	`/api/cameras`	Cadastra uma câmera	201, 400, 404, 500
PUT	`/api/cameras/{id}`	Atualiza uma câmera	200, 400, 404, 500
DELETE	`/api/cameras/{id}`	Exclui uma câmera	204, 400, 404, 500
Ocorrências			
GET	`/api/ocorrencias?status=&ativas=`	Lista as ocorrências (filtros opcionais por status ou só ativas)	200, 400, 500
GET	`/api/ocorrencias/{id}`	Busca uma ocorrência pelo id	200, 400, 404, 500

Verbo	URI	Descrição	Status
POST	`/api/ocorrencias`	Registra um alagamento (relato do cidadão ou operador)	201, 400, 404, 500
PUT	`/api/ocorrencias/{id}`	Atualiza uma ocorrência	200, 400, 404, 500
PUT	`/api/ocorrencias/{id}/validacao`	Confirma, descarta ou resolve a ocorrência (gera/encerra alerta)	200, 400, 404, 500
DELETE	`/api/ocorrencias/{id}`	Exclui uma ocorrência	204, 400, 404, 500
Alertas			
GET	`/api/alertas?vigentes=`	Lista os alertas (todos ou só os em vigor)	200, 500
GET	`/api/alertas/{id}`	Busca um alerta pelo id	200, 400, 404, 500
POST	`/api/alertas`	Emite um alerta manual	201, 400, 404, 500
POST	`/api/alertas/previsao?simulacao=`	Gera alertas para as regiões com risco alto ou crítico	200, 201, 502, 500
PUT	`/api/alertas/{id}`	Atualiza (ou encerra) um alerta	200, 400, 404, 500
DELETE	`/api/alertas/{id}`	Exclui um alerta	204, 400, 404, 500
Rotas seguras			
GET	`/api/rotas`	Histórico de rotas calculadas	200, 500
GET	`/api/rotas/{id}`	Busca uma rota com o trajeto completo	200, 400, 404, 500
POST	`/api/rotas`	Calcula a rota que desvia dos alagamentos e salva	201, 400, 502, 500
PUT	`/api/rotas/{id}`	Recalcula uma rota salva com os alagamentos atuais	200, 400, 404, 502, 500
DELETE	`/api/rotas/{id}`	Exclui uma rota	204, 400, 404, 500

200 sucesso, 201 criado, 204 excluído, 400 dados inválidos, 404 não encontrado, 502 serviço externo indisponível, 500 erro no banco. Os erros retornam `{"status": 400, "erro": "mensagem"}`.

6.1 Exemplos de requisição

POST /api/rotas

```
{ "origem": "Praça da Sé", "origemLat": -23.5503, "origemLon": -46.6339,
  "destino": "Shopping Ibirapuera", "destinoLat": -23.61, "destinoLon": -46.666 }

// resposta 201 (resumida)
{ "rota": { "id": 8, "situacao": "SEGURA", "distanciaKm": 9.2, "duracaoMin": 17.9,
            "distanciaDiretaKm": 9.4, "duracaoDiretaMin": 13.9, "alagamentosEvitados": 1, ... },
  "alagamentosNoCaminhoDireto": [ { "id": 3, "endereco": "Av. 23 de Maio - altura do Viaduto Tutoia", ...
} ],
  "alternativasAvaliadas": 4,
  "mensagem": "Rota segura encontrada: desvia de 1 alagamento(s) com 4.0 min a mais de viagem." }
```

POST /api/monitoramento/analises

```
{ "idCamera": 10, "urlImagem": "https://cameras.cetsp.com.br/cams/23/7.jpg", "registrarOcorrencia": true
}

// resposta 200 (sem alagamento) ou 201 (alagamento registrado)
{ "analise": { "alagamentoDetectado": false, "nivelAguaEstimadoCm": 0, "nivelRisco": "BAIXO",
               "confianca": 85.0, "viaTransitavel": true, "origem": "openai", ... },
  "ocorrencia": null,
  "mensagem": "Nenhum alagamento identificado na imagem." }
```

PUT /api/ocorrencias/{id}/validacao

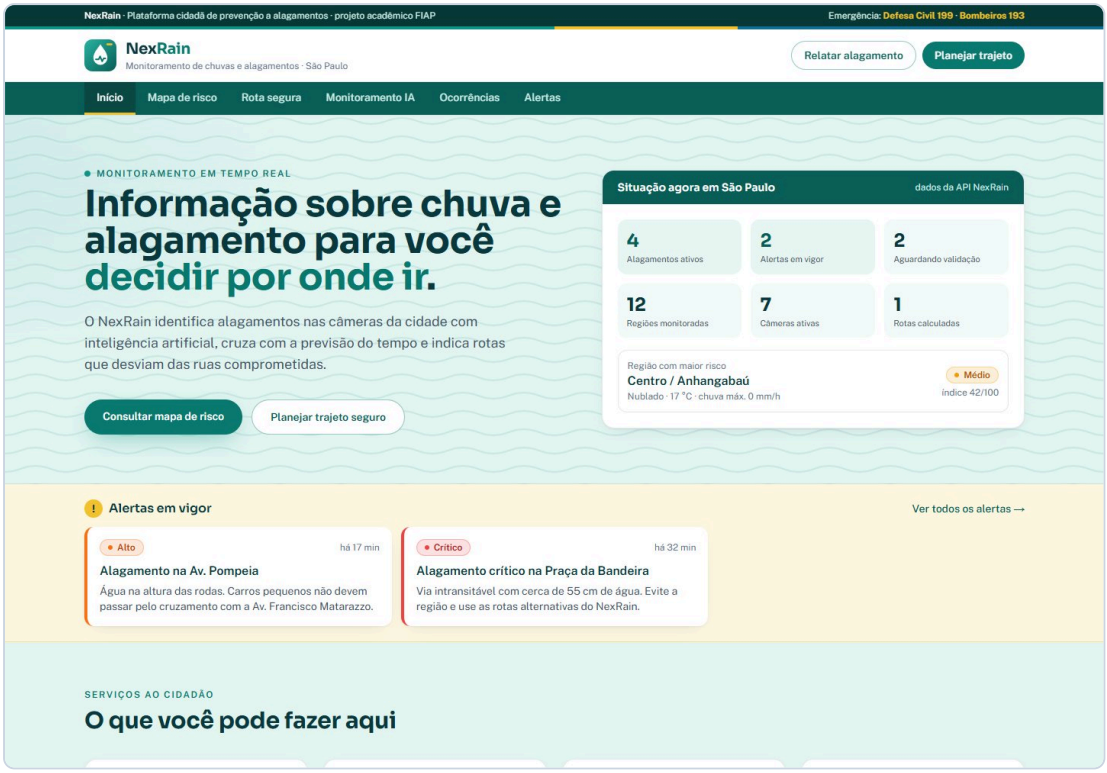
```
{ "acao": "CONFIRMAR", "responsavel": "Defesa Civil - Operador 07" }

// confirmar uma ocorrência que não está pendente retorna 400
{ "status": 400, "erro": "Só é possível confirmar uma ocorrência pendente (status atual: CONFIRMADA)" }
```

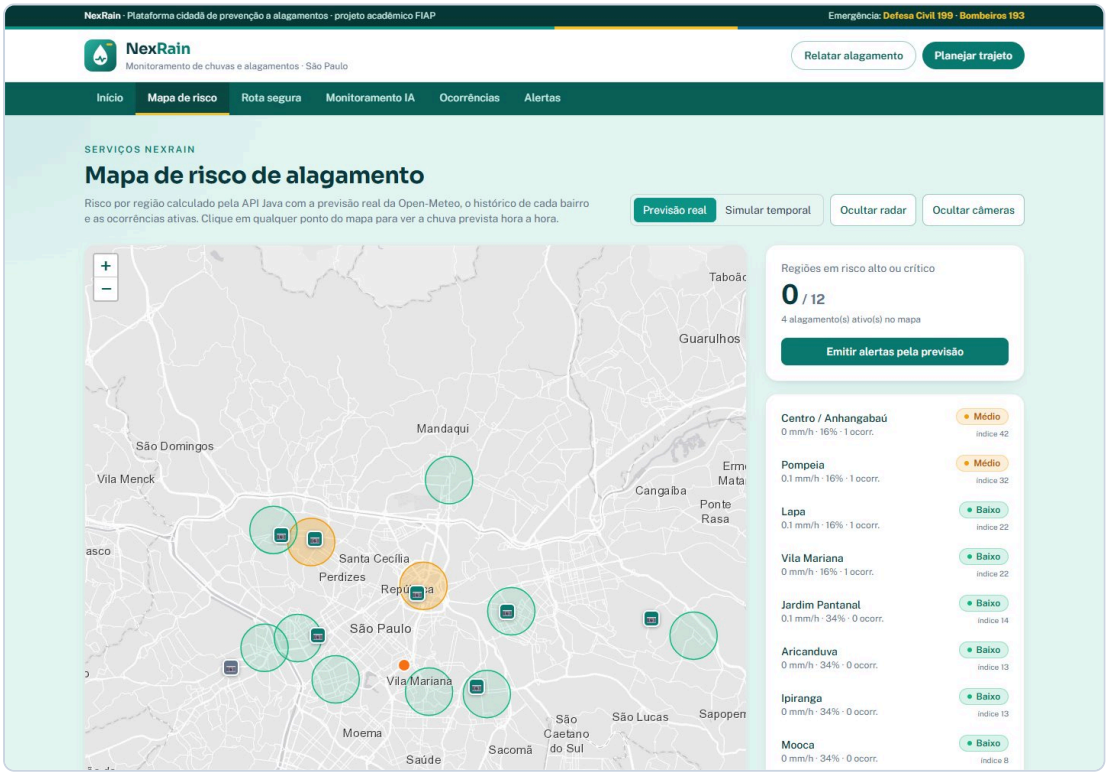
POST /api/ocorrencias

```
{ "endereco": "Av. Prestes Maia", "latitude": -23.5418, "longitude": -46.6355, "nivelAguaCm": 60,
  "descricao": "Água acima da roda dos carros" }
// sem idRegiao a API usa a região mais próxima; sem nivelRisco ele é calculado pela altura da água
```

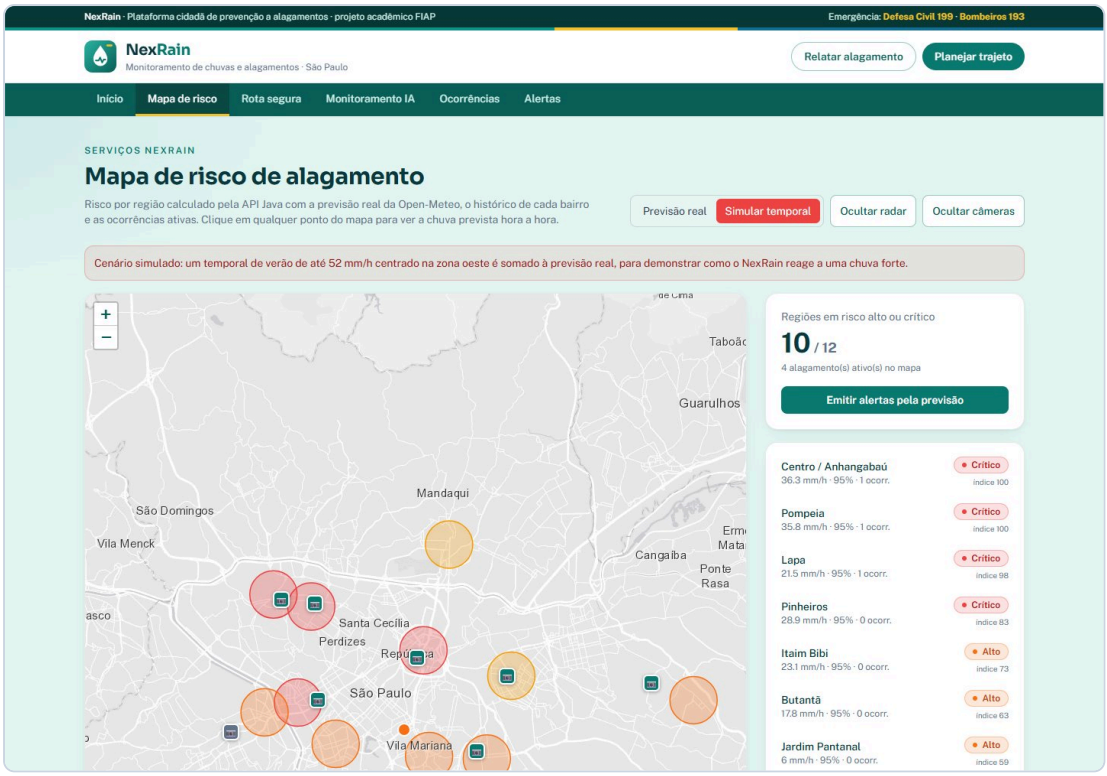
7. Protótipos de Tela



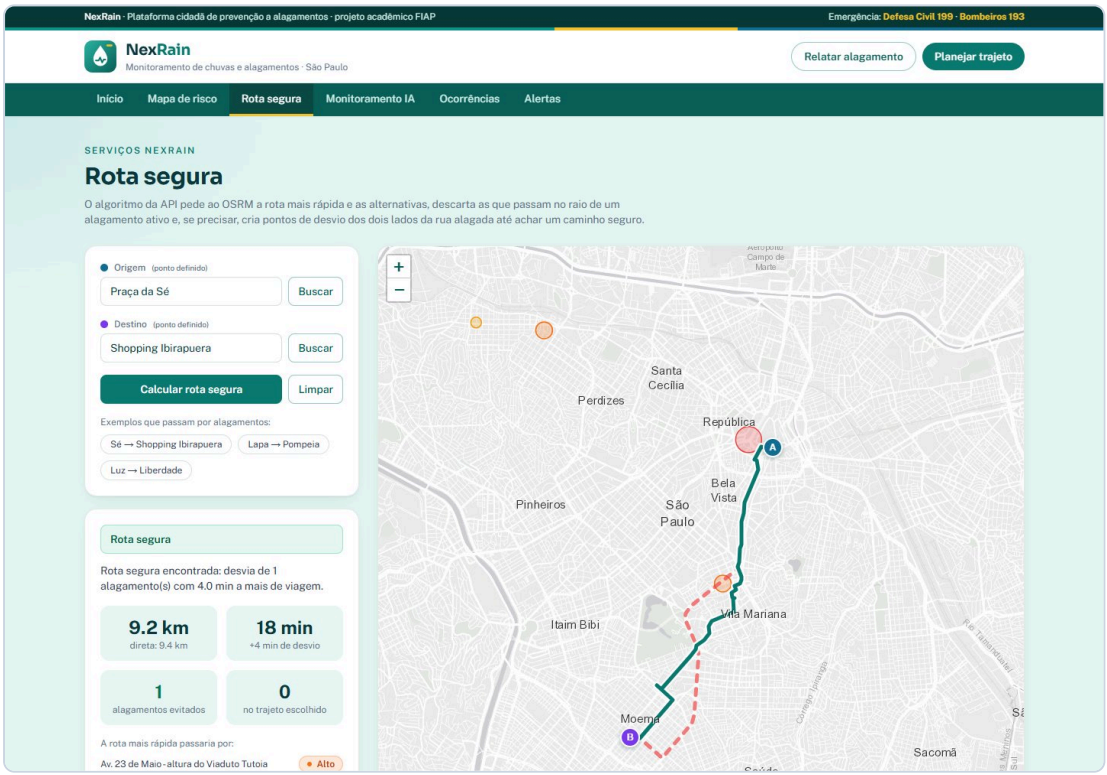
Página inicial - `GET /api/painel`, `GET /api/alertas?vigentes=true` e `GET /api/riscos`.



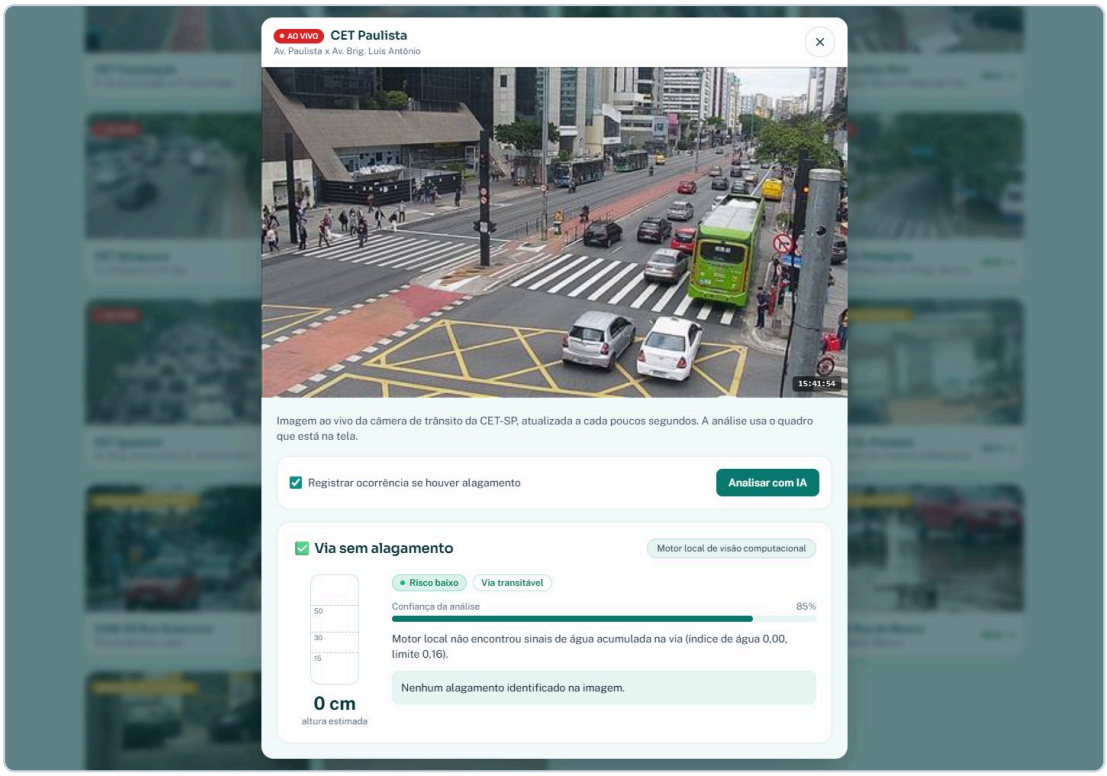
Mapa de risco - `GET /api/riscos`, `GET /api/ocorrencias?ativas=true`, `GET /api/cameras` e `GET /api/clima`.



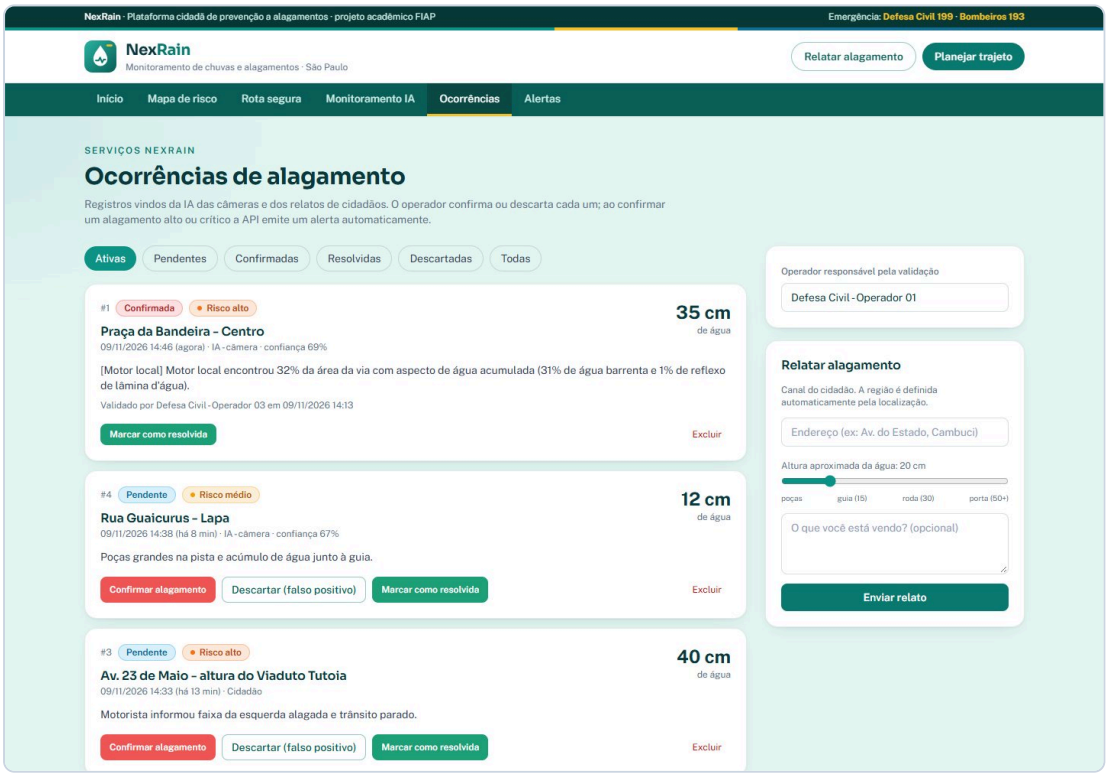
Mapa de risco com chuva simulada - `GET /api/riscos?simulacao=true` e `POST /api/alertas/previsao`.



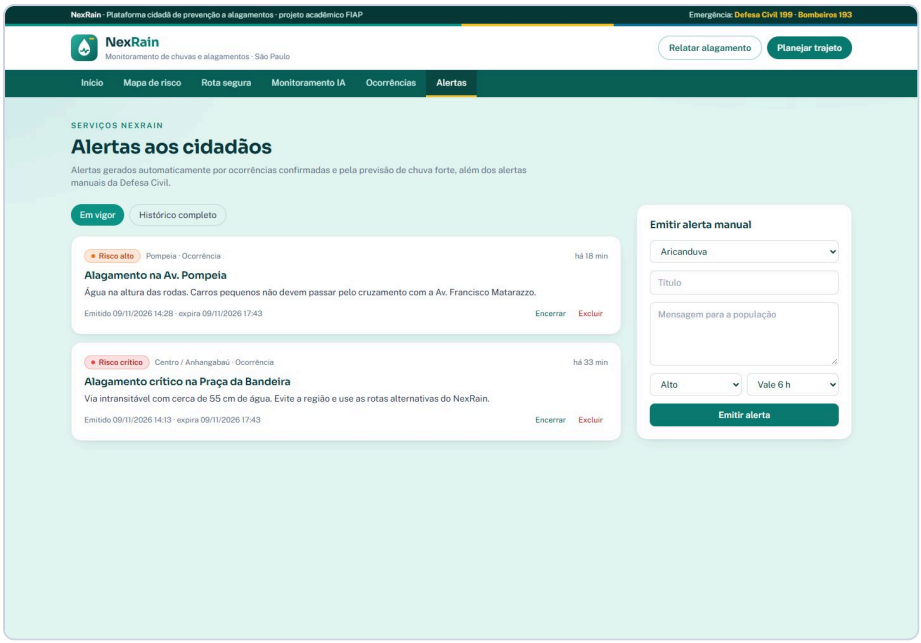
Rota segura - `POST /api/rotas`, `GET /api/rotas`, `PUT /api/rotas/{id}` e `DELETE /api/rotas/{id}`. Em vermelho a rota mais rápida, que passa pelo alagamento da Av. 23 de Maio, e em verde a rota calculada pela API.



Monitoramento por IA - `GET /api/cameras` e `POST /api/monitoramento/analises` com a imagem ao vivo da câmera da CET-SP.



Ocorrências - `GET /api/ocorrencias`, `POST /api/ocorrencias`, `PUT /api/ocorrencias/{id}/validacao` e `DELETE /api/ocorrencias/{id}`.



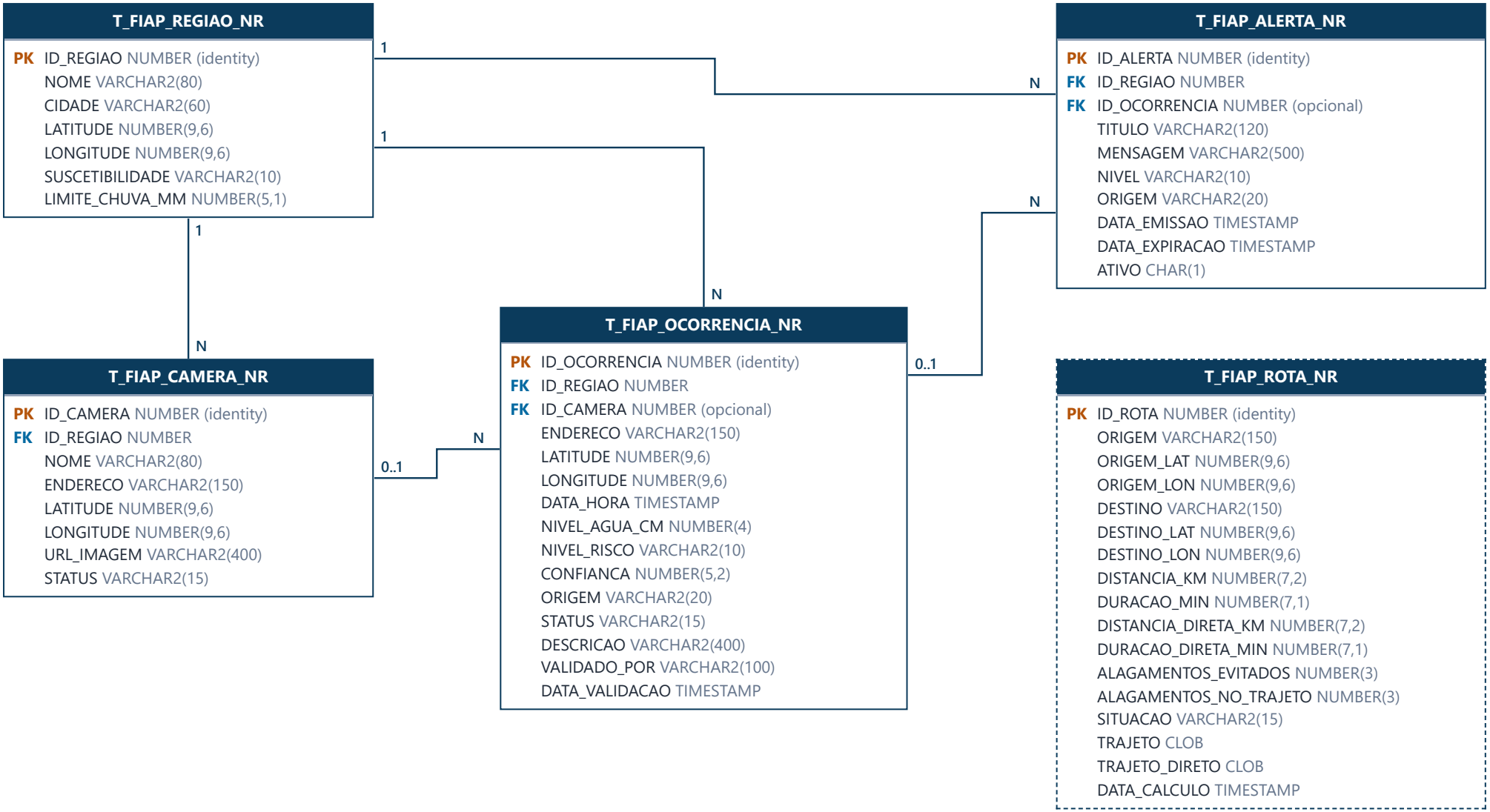
Alertas - `GET /api/alertas`, `POST /api/alertas`, `PUT /api/alertas/{id}` e `DELETE /api/alertas/{id}`.



Rota segura no celular.

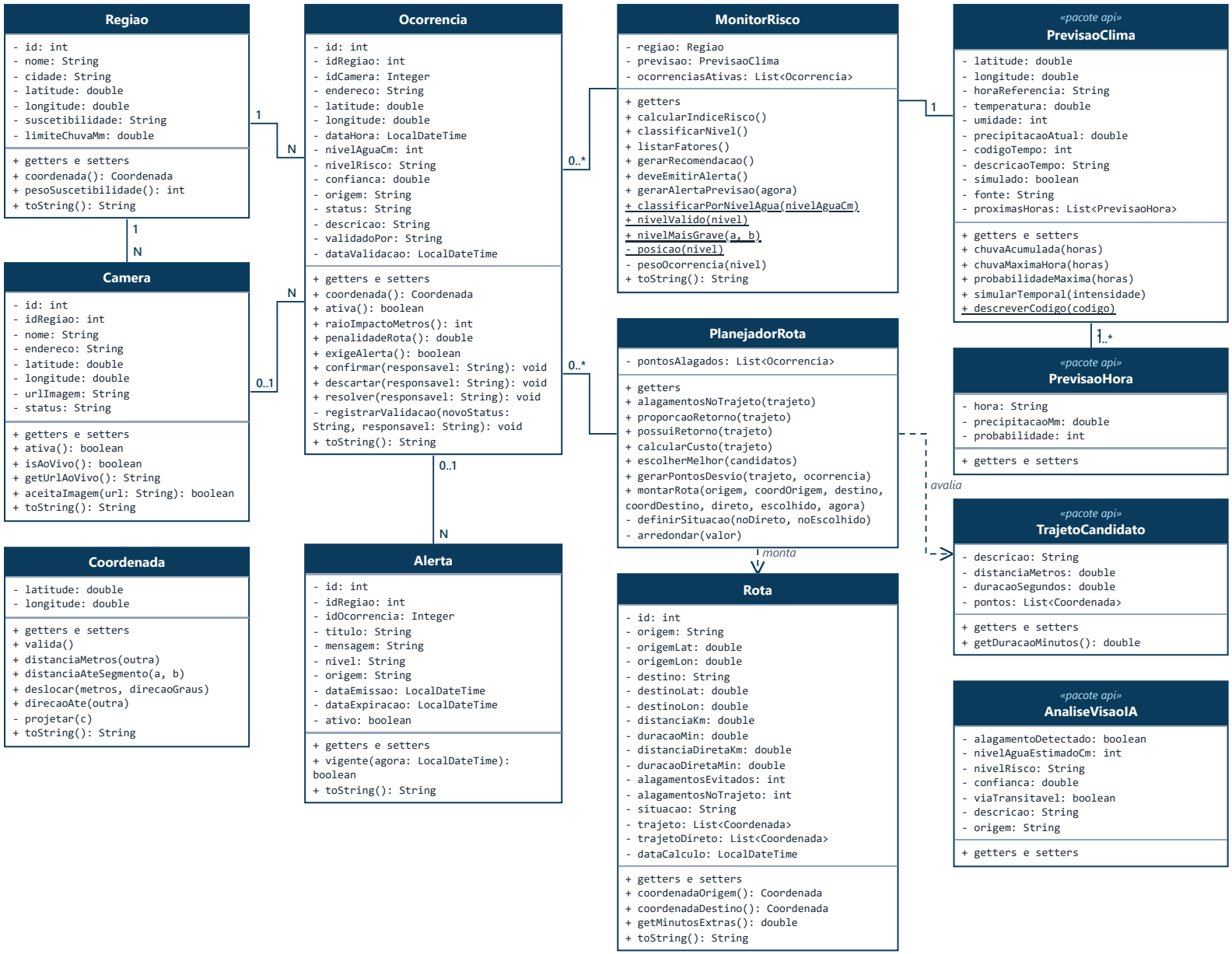
8. Modelo do Banco de Dados (DER)

Cinco tabelas no Oracle. Uma região tem várias câmeras, ocorrências e alertas; uma câmera pode ter várias ocorrências; uma ocorrência pode ter alertas. A tabela de rotas guarda o histórico das rotas calculadas e não tem chave estrangeira.



9. Diagrama de Classes

- Camada Model (pacotes `entities` e `api`). `Coordenada` também é usada por `Rota` e `TrajetoCandidato`; `MonitorRisco.gerarAlertaPrevisao()` cria um `Alerta`.



9. Diagrama de Classes

- DTOs e exceções

«pacote dto»
EntradaAnaliseImagem

```
- idCamera: Integer
- imagemBase64: String
- urlImagem: String
- idRegiao: Integer
- endereco: String
- latitude: Double
- longitude: Double
- registrarOcorrencia: boolean
```

```
+ getters e setters
```

«pacote dto»
ResultadoAnaliseImagem

```
- analise: AnaliseVisaoIA
- origemAnalise: String
- camera: Camera
- ocorrencia: Ocorrencia
- mensagem: String
```

```
+ getters e setters
```

«pacote dto»
RiscoRegiao

```
- regiao: Regiao
- indice: int
- nivel: String
- fatores: List<String>
- recomendacao: String
- ocorrenciasAtivas: int
- precipitacaoAtual: double
- chuvaMaxima6h: double
- chuvaAcumulada3h: double
- probabilidadeChuva: int
- temperatura: double
- descricaoTempo: String
- simulado: boolean
```

```
+ getters
```

«pacote dto»
SolicitacaoRota

```
- origem: String
- origemLat: Double
- origemLon: Double
- destino: String
- destinoLat: Double
- destinoLon: Double
```

```
+ getters e setters
```

«pacote dto»
ResultadoRota

```
- rota: Rota
- alagamentosNoCaminhoDireto:
List<Ocorrencia>
- alagamentosNoTrajeto: List<Ocorrencia>
- alternativasAvaliadas: int
- mensagem: String
```

```
+ getters
```

«pacote dto»
ValidacaoOcorrencia

```
- acao: String
- responsavel: String
```

```
+ getters e setters
```

«pacote dto»
LocalEncontrado

```
- nome: String
- latitude: double
- longitude: double
```

```
+ getters
```

«pacote dto»
PainelResumo

```
- regioesMonitoradas: int
- camerasAtivas: int
- ocorrenciasAtivas: int
- ocorrenciasPendentes: int
- alertasAtivos: int
- rotasCalculadas: int
```

```
+ getters
```

«pacote api»
RespostaOpenAi

```
- choices: List<Escolha>
```

```
+ getters e setters
+ getConteudo(): String
```

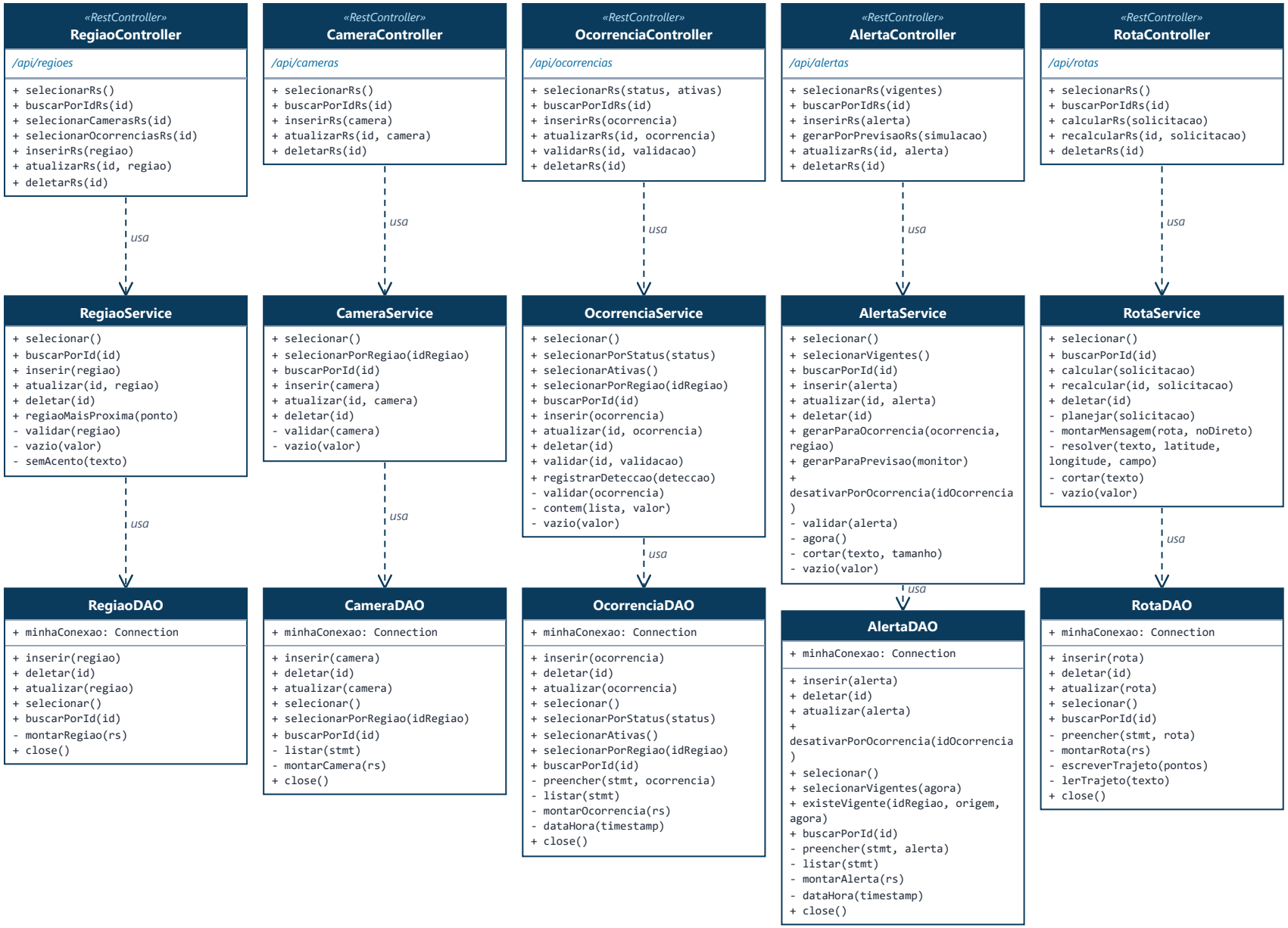
«pacote excecoes»
RegraNegocioException

«pacote excecoes»
RecursoNaoEncontradoException

«pacote excecoes»
ServicoExternoException

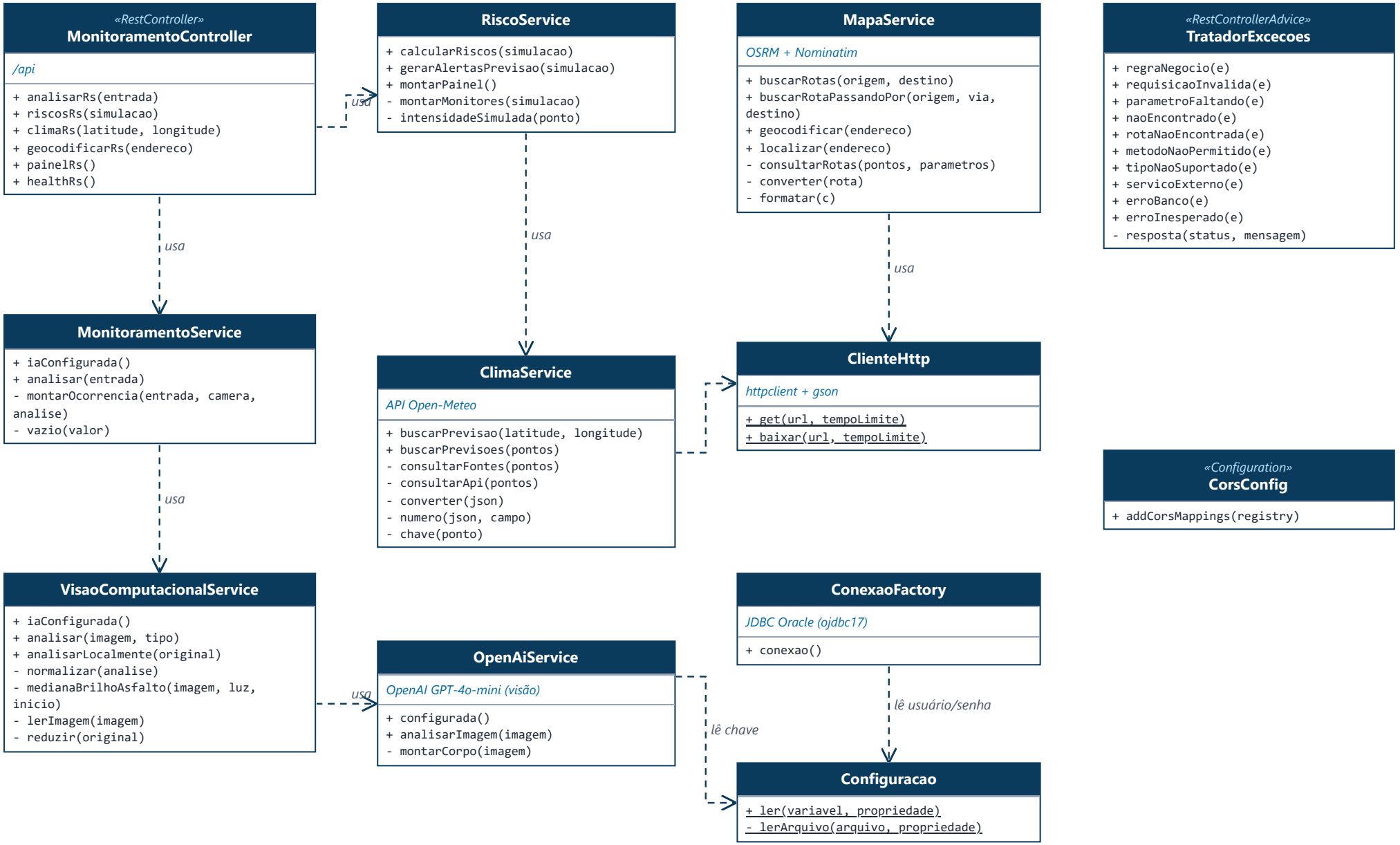
9. Diagrama de Classes

- Camadas Controller, Service e DAO



9. Diagrama de Classes

- Serviços de integração e conexões



10. Como Executar

Ferramentas e versões

Item	Versão
Java JDK	21
Spring Boot	4.1.1
Maven	3.9 (já incluso no projeto pelo `mvnw`)
Banco de dados	Oracle da FIAP (`oracle.fiap.com.br:1521:orcl`), driver ojdbc17
Bibliotecas	Apache HttpClient 4.5.14 e Gson
Ferramentas	IntelliJ IDEA, SQL Developer ou DBeaver e Postman

Links

GitHub	https://github.com/Tuco2006/nexrain-springboot
API publicada (Render)	https://nexrain-springboot.onrender.com/api/health
Site publicado	https://nexrain-front.vercel.app/

A API usa o plano gratuito do Render, que desliga depois de um tempo sem uso. O primeiro acesso pode levar cerca de um minuto.

Passo a passo

1. **Banco:** executar o arquivo `script.sql` inteiro no Oracle da FIAP (F5 no SQL Developer ou Alt+X no DBeaver). Ele cria as 5 tabelas e cadastra 12 regiões, 18 câmeras, 4 ocorrências e 2 alertas. Para recriar o banco, rodar antes o `drop.sql`.
2. **Configuração:** criar o arquivo `src/main/resources/local.properties` com `db.usuario`, `db.senha` e, se for usar a OpenAI, `openai.api.key`. Esse arquivo não vai para o GitHub.
3. **API:** abrir o projeto no IntelliJ e rodar a classe `NexrainApplication`, ou no terminal `.\mvnw spring-boot:run`. A API sobe em `http://localhost:8080`.
4. **Teste:** acessar `http://localhost:8080/api/health` e testar os endpoints da seção 6 pelo Postman.

Publicação

A API roda no Render usando o `Dockerfile` do projeto (New > Web Service, linguagem Docker, Health Check Path `/api/health`), com as variáveis `DB_USUARIO`, `DB_SENHA` e `OPENAI_API_KEY`.